

Semester	Jahr	Abgabedatum	Vortragsdatum
Sommersemester	26	30.07.2026	31.07.2026

Vorlesung: Innovative Konzepte zur Programmierung von Industrierobotern **Dozent:** Prof. Dr.-Ing. Björn Hein
 Projektaufgabe - RRT mit kantenbewusster Erweiterung

Ziel der Projektaufgabe ist die Analyse und Überarbeitung eines RRT-Ansatzes. Die vorhandene Implementierung in `IPRRT.py` verwendet für die Erweiterung des Baums im Wesentlichen den nächsten Knoten zum zufällig gewählten Sample. Der geometrische Graph, insbesondere die bereits vorhandenen Kanten des Baums, wird dabei nicht als kontinuierlich eingebettete Struktur betrachtet.

In dieser Aufgabe soll eine methodisch sauberere Variante entwickelt und evaluiert werden: Für ein neues Sample soll nicht nur der nächste Knoten, sondern der nächste Punkt auf dem bestehenden Baum betrachtet werden. Dieser Punkt kann ein vorhandener Knoten sein oder eine Projektion auf eine vorhandene Kante. Liegt der nächste Punkt auf einer Kante, muss die Kante geeignet aufgeteilt werden, bevor der Baum weiter erweitert wird.

Der Ansatz soll nicht auf 2-DoF beschränkt sein. Die geometrischen Operationen im Konfigurationsraum, insbesondere Abstand, Projektion, Interpolation und Kantenaufteilung, sollen so formuliert werden, dass sie für n-DoF-Konfigurationen funktionieren.

Die zentrale Abgabe ist ein Jupyter Notebook. Dieses Notebook dient als technischer Bericht: Es soll Vorgehensweise, verwendete Module, Experimente, Visualisierungen, Ergebnisse und Diskussion nachvollziehbar enthalten.

Der eigentliche Python-Code soll nach Möglichkeit in Modulen implementiert und im Notebook importiert werden. Das Notebook soll nicht aus langen Codeblöcken bestehen, sondern die Lösung strukturiert erklären, ausführen und auswerten.

Ausgangspunkt sind insbesondere:

- `notebooks/IPRRT.py`
- `notebooks/IPVISRRT.py`
- `notebooks/IPEnvironment.py`
- `notebooks/IPEnvironmentKin.py`
- `notebooks/IPPlanarManipulator.py`
- `notebooks/IPPerfMonitor.py`
- `notebooks/IP-4-0-Test-Environments.ipynb`
- `notebooks/IP-9-0-RRT-Basics.ipynb`
- `notebooks/IP-10-0-PlanarManipulator.ipynb`
- `notebooks/IP-X-0-Benchmarking-concept.ipynb`
- `notebooks/IP-X-1-Automated_PlanerTest.ipynb`

I.) Erwartete Notebook-Struktur

1. Einleitung und Zielsetzung

Beschreiben Sie,

1. wie ein RRT grundsätzlich aufgebaut wird,
2. welche Rolle Sampling, Nächster-Nachbar-Suche, lokale Erweiterung und Kollisionsprüfung spielen,
3. warum die Betrachtung nur der vorhandenen Knoten eine Vereinfachung darstellt,
4. was sich ändert, wenn der Baum als geometrischer Graph aus Knoten und Kanten betrachtet wird.

Diskutieren Sie dabei sauber: Der klassische einfache RRT kann über nächste Knoten formuliert werden. In dieser Aufgabe soll aber untersucht werden, ob eine kantenbewusste Variante den geometrischen Baum besser ausnutzt.

2. Analyse der vorhandenen Implementierung

Analysieren Sie `IPRRT.py` und dokumentieren Sie:

1. wie Start und Ziel geprüft werden,
2. wie neue Samples erzeugt werden,

3. wie der nächste Knoten mit `ckdTree` bestimmt wird,
4. wie neue Knoten und Kanten in `planner.graph` angelegt werden,
5. wie oft und an welcher Stelle das Ziel getestet wird,
6. welche Unterschiede zwischen `RRTSimple` und `RRT` bestehen,
7. warum vorhandene Kanten bei der Auswahl des Erweiterungspunkts bisher keine Rolle spielen.

Halten Sie fest, welche Teile der bestehenden Implementierung übernommen werden können und welche Teile ersetzt werden müssen.

3. Geometrisches Konzept: nächster Punkt auf dem Baum

Entwickeln Sie ein Verfahren, das zu einem Sample `q_rand` den nächsten Punkt auf dem vorhandenen Baum bestimmt.

Zu berücksichtigen sind:

1. vorhandene Knoten,
2. vorhandene Kanten als Liniensegmente im Konfigurationsraum,
3. Projektion von `q_rand` auf ein Segment,
4. Begrenzung der Projektion auf das Segment,
5. Abstand zwischen `q_rand` und nächstem Punkt,
6. Sonderfälle am Anfang oder Ende einer Kante.

Dokumentieren Sie die verwendete Mathematik. Für eine Kante zwischen `q_a` und `q_b` soll die Projektion auf das Segment berechnet und auf den Parameterbereich `[0, 1]` beschränkt werden.

Die Berechnung soll für beliebige Dimensionen formuliert werden und nicht nur für zweidimensionale Punkte funktionieren.

4. Aufteilen von Kanten

Wenn der nächste Punkt auf einer vorhandenen Kante liegt, soll die Kante aufgeteilt werden.

Implementieren Sie:

1. Erzeugen eines neuen Zwischenknotens auf der Kante,
2. Entfernen der alten Kante,
3. Hinzufügen zweier neuer Kanten zu den bisherigen Endknoten,
4. Übernahme bzw. Neuberechnung von Kantengewichten,
5. Sicherstellung, dass der Baum zusammenhängend und kreisfrei bleibt.

Diskutieren Sie, ob vor dem Aufteilen der Kante erneut eine Kollisionsprüfung notwendig ist. Begründen Sie Ihre Entscheidung.

5. RRT-Erweiterung vom nächsten Baumpunkt

Implementieren Sie eine RRT-Erweiterung, die vom nächsten Punkt auf dem Baum in Richtung `q_rand` erweitert.

Anforderungen:

1. konfigurierbare Schrittweite `eta`,
2. Erweiterung höchstens bis `q_rand`,
3. Kollisionsprüfung der neuen Verbindung,
4. Einfügen des neuen Knotens nur bei kollisionsfreier Verbindung,
5. optionales direktes Verbinden mit dem Ziel,
6. Unterstützung von n-DoF-Konfigurationen,

7. sinnvolle Rückgabe eines Pfads über `nx.shortest_path`.

Der neue Planer soll weiterhin eine ähnliche Schnittstelle wie die vorhandenen Planer besitzen:

1. `planPath(startList, goalList, config)`,
2. Startkonfiguration in `startList`,
3. Zielkonfiguration in `goalList`,
4. Parameter über `config`.

6. Vergleichsvarianten

Implementieren oder verwenden Sie mindestens zwei Varianten:

1. vorhandener knotenbasierter RRT aus `IPRRT.py`,
2. neue kantenbewusste RRT-Variante.

Optional können Sie ergänzen:

1. Goal Bias,
2. bidirektionale Variante,
3. RRT-Connect-artige Erweiterung,
4. einfache Nachglättung des gefundenen Pfads.

Zusätzliche Varianten sind nur sinnvoll, wenn die Grundvariante korrekt implementiert und ausgewertet wurde.

7. Visualisierung

Erweitern Sie die RRT-Visualisierung so, dass die Unterschiede sichtbar werden.

Visualisieren Sie mindestens:

1. Baumknoten,
2. Baumkanten,
3. zufällig gewählte Samples,
4. nächsten Knoten der einfachen Variante,
5. nächsten Punkt auf einer Kante der kantenbewussten Variante,
6. eingefügte Zwischenknoten,
7. finalen Lösungspfad.

Zeigen Sie mindestens ein kleines Beispiel, in dem der nächste Punkt auf einer Kante und nicht auf einem vorhandenen Knoten liegt.

8. Evaluation mit 2-DoF-Punktrobotern

Evaluieren Sie die Varianten anhand von mindestens drei 2-DoF-Benchmarkumgebungen.

Die Umgebungen sollen unterschiedliche Eigenschaften besitzen:

1. offene Umgebung,
2. Engstelle,
3. fallenartige oder stark zerklüftete Umgebung.

Variieren Sie:

1. maximale Anzahl erzeugter Knoten,
2. Schrittweite `eta`,
3. Zieltest-Frequenz,
4. optional Goal-Bias-Wahrscheinlichkeit.

Führen Sie pro Benchmark und Parameterkombination mindestens 30 Planungsläufe durch.

9. Evaluation mit n-DoF-PlanarManipulator

Evaluieren Sie den Ansatz zusätzlich anhand von mindestens zwei PlanarManipulator-Beispielen im n-DoF-Konfigurationsraum.

Nutzen Sie die Beispiele aus `IP-10-0-PlanarManipulator.ipynb` und `IPEnvironmentKin.py`.

Betrachten Sie mindestens:

1. 2-DoF-PlanarManipulator,
2. 4-DoF-PlanarManipulator,
3. optional 6-DoF-PlanarManipulator als Vertiefung.

Diskutieren Sie, ob die Projektion auf Kanten im Konfigurationsraum für den PlanarManipulator genauso interpretiert werden kann wie beim 2-DoF-Punktroboter.

10. Statistik und Vergleich

Erfassen Sie mindestens:

1. Erfolgsrate,
2. Planungszeit,
3. Anzahl erzeugter Knoten,
4. Anzahl Kanten,
5. Anzahl aufgeteilter Kanten,
6. Anzahl Kollisionsprüfungen,
7. Pfadlänge,
8. Anzahl Pfadsegmente,
9. mittlere Distanz vom Sample zum gewählten Erweiterungspunkt,
10. Anteil der Erweiterungen, die von einer Kante statt von einem vorhandenen Knoten starten.

Stellen Sie die Ergebnisse in Tabellen und Diagrammen dar.

11. Validierung der Korrektheit

Prüfen Sie Ihre Implementierung gezielt.

Mindestens erwartet:

1. Unit-Test für die Projektion eines Punkts auf ein Segment,
2. Unit-Test für Projektionen außerhalb des Segments,
3. Unit-Test für das Aufteilen einer Kante,
4. Prüfung, dass der Graph nach dem Aufteilen zusammenhängend bleibt,
5. Prüfung, dass keine ungewollten Zyklen entstehen,
6. Prüfung, dass alle eingefügten Kanten kollisionsfrei sind,
7. Vergleich des gefundenen Pfads mit der Visualisierung.

12. Diskussion und Fazit

Beantworten Sie:

1. Was ist der wesentliche Unterschied zwischen knotenbasierter und kantenbewusster Erweiterung?
2. In welchen Umgebungen bringt die kantenbewusste Variante Vorteile?
3. Wann entstehen keine oder nur geringe Vorteile?

4. Welche zusätzlichen Kosten entstehen durch die Suche über Kanten?
5. Wie kann die Kantensuche effizienter implementiert werden?
6. Wie beeinflussen **eta**, Zieltest-Frequenz und Goal Bias das Ergebnis?
7. Welche Variante würden Sie für weitere Bahnplanungsaufgaben weiterverwenden?

II.) Präsentation

Zusätzlich zur Notebook-Abgabe gibt es eine Präsentation. Sie können dafür entweder das Notebook direkt als Präsentationsgrundlage verwenden oder daraus separate Folien erzeugen.

Der Vortragstag ist der 31.07.2026.

Die Präsentation soll nicht den gesamten Code erklären, sondern die wichtigsten Entscheidungen, Visualisierungen, Messergebnisse und Erkenntnisse herausarbeiten.

III.) Verwendung von KI-Werkzeugen

Sie dürfen Codex oder andere KI-Werkzeuge verwenden.

Die Bewertung erfolgt nicht danach, ob Code vollständig ohne Hilfsmittel geschrieben wurde, sondern danach, ob Sie die Lösung verstehen, integrieren, testen und kritisch bewerten können.

Dokumentieren Sie kurz:

1. wofür KI verwendet wurde,
2. welche Vorschläge übernommen wurden,
3. welche Vorschläge verworfen wurden,
4. wie Sie geprüft haben, dass der erzeugte Code korrekt funktioniert.

IV.) Abgabe

Abzugeben sind:

1. Jupyter Notebook als technischer Bericht,
2. Python-Module mit der kantenbewussten RRT-Variante,
3. angepasste oder ergänzte Visualisierungsmodule,
4. Tests für Projektion, Kantenaufteilung und Kollisionsfreiheit,
5. Benchmarkauswertungen,
6. erzeugte Abbildungen/Animationen, sofern sie nicht direkt im Notebook enthalten sind,
7. optional separate Präsentationsfolien.

Anmerkung: Bitte checken Sie die Notebooks „IP-X-0-Benchmarking-concept.ipynb“ und „IP-X-1-Automated_PlanerTest.ipynb“ für Profiling und Statistiken.

Viel Erfolg!